

Information Theory Final Project

Range Coding and a Simplified DEFLATE Compressor

1 Project Overview

In this project, you will implement two different lossless compression methods.

The first method is a simplified arithmetic/range coding compressor. The purpose of this part is to understand how a file can be compressed by repeatedly narrowing an integer range according to symbol frequencies.

The second method is a simplified compressor inspired by the main ideas inside DEFLATE. This is the same DEFLATE-style compressor described in the original project.

The project has two main parts:

Part 1: Simplified Arithmetic/Range Coding

and:

Part 2: Simplified DEFLATE Compressor

Both compressors must work on files as byte streams. Even if the file is a text file, your program should treat it as bytes, not as Python strings.

For both parts, the final correctness requirement is:

$\text{decompress}(\text{compress}(\text{data})) = \text{data}$

byte-for-byte.

At the end of the project, you must submit your implementation and a short report comparing the two compression methods on different files.

2 Part 1: Simplified Arithmetic/Range Coding

2.1 Purpose

Arithmetic coding and range coding compress data using symbol probabilities.

Instead of assigning a separate bit code to each symbol like Huffman coding, arithmetic coding represents the whole message by one interval.

For example, suppose the alphabet is:

Symbol	Count	Range
A	5	[0, 5)
B	3	[5, 8)
C	2	[8, 10)

The total count is:

10

So symbol *A* gets the first half of the range, *B* gets the next part, and *C* gets the final part.

The encoder starts with a large interval. Each symbol narrows the interval. The decoder performs the same narrowing process and recovers the original symbols.

The important idea is:

The encoder and decoder must use the same frequency table.

In this project, the frequency table is written at the beginning of the compressed file. After that, the encoded payload is written.

2.2 Input and Output

The input to Part 1 is the original byte array:

$$b_0, b_1, b_2, \dots, b_{n-1}$$

where each byte is between 0 and 255.

The output is a compressed file containing:

Frequency Table + Range-Coded Payload

There is no extra magic header for the range-coded file.

The decompressor knows that the file is a range-coded file because it is opened using the range-decoding command. It first reads the frequency table, then reads the encoded payload.

The final correctness requirement is:

`range_decompress(range_compress(data)) = data`

byte-for-byte.

2.3 Alphabet

Use an alphabet of 257 symbols:

$$0, 1, 2, \dots, 255, 256$$

Symbols 0–255 are normal byte values.

Symbol 256 is a special end-of-file symbol:

$$\text{EOF} = 256$$

The EOF symbol tells the decoder when to stop decoding.

For example, if the input file contains **ABC**, then the encoder actually encodes:

$$65, 66, 67, 256$$

where 65, 66, and 67 are the byte values of **A**, **B**, and **C**.

The original file size does not need to be stored, because EOF tells the decoder when the file ends.

2.4 Frequency Table

Before encoding, count how many times each byte appears in the file.

Then add:

$$freq[256] = 1$$

for EOF.

For example, if the input is **banana**, then the frequencies are:

Symbol	Frequency
a	3
b	1
n	2
EOF	1

All other byte values have frequency 0.
The total frequency is:

$$total = \sum_{s=0}^{256} freq[s]$$

For **banana**, the total is:

$$7$$

because the file has 6 bytes and one EOF symbol.

2.5 Low-Level Frequency Table Format

The range-coded compressed file starts directly with the frequency table.

The frequency table contains exactly 257 frequency values:

$$freq[0], freq[1], freq[2], \dots, freq[255], freq[256]$$

Each frequency value must be written using 4 bytes.

Therefore, the frequency table size is:

$$257 \times 4 = 1028 \text{ bytes}$$

The order is fixed:

Position in Table	Meaning
0	Frequency of byte value 0
1	Frequency of byte value 1
2	Frequency of byte value 2
$\vdots$	$\vdots$
255	Frequency of byte value 255
256	Frequency of EOF symbol

Each frequency must be stored as an unsigned 32-bit integer.

Use big-endian order for these 4 bytes.

That means the most significant byte is written first.

For example, the number:

$$258$$

is:

$$00000000 \ 00000000 \ 00000001 \ 00000010$$

so it is written as the four bytes:

$$0, 0, 1, 2$$

After the 1028-byte frequency table, the range-coded payload starts immediately.

So the file layout is:

Field	Size	Description
FREQ_TABLE	1028 bytes	257 unsigned 32-bit frequencies in big-endian order
PAYLOAD	variable	Range-coded bitstream

2.6 Cumulative Ranges

The frequency table is converted into cumulative ranges.

For example, suppose we have only four symbols:

$$a, b, n, EOF$$

with frequencies:

$$freq(a) = 3$$

$$freq(b) = 1$$

$$freq(n) = 2$$

$$freq(EOF) = 1$$

The total is:

$$7$$

The cumulative ranges are:

Symbol	Frequency	Cumulative Low	Cumulative High
a	3	0	3
b	1	3	4
n	2	4	6
EOF	1	6	7

So:

$$a \rightarrow [0, 3)$$

$$b \rightarrow [3, 4)$$

$$n \rightarrow [4, 6)$$

$$EOF \rightarrow [6, 7)$$

The decoder does not need the cumulative ranges to be stored directly.

It reads the frequency table, then rebuilds the same cumulative ranges by summing the frequencies in symbol order.

2.7 Why We Do Not Use Floating Point

Arithmetic coding is often explained using intervals such as:

$$[0.37, 0.40)$$

However, real files can contain thousands or millions of bytes. The interval becomes extremely small.

If floating point numbers are used, eventually the interval may become too small to represent accurately.

For example, because of rounding, the computer may reach a situation where:

$$low = high$$

even though mathematically the interval should still have a tiny positive width. Therefore, your implementation must not use floating point numbers for the actual compression. Use integer arithmetic.

2.8 Integer Interval

In this project, use a 32-bit integer interval:

$$[low, high]$$

The initial interval is:

$$low = 0$$

$$high = 2^{32} - 1$$

For each symbol, compute:

$$range = high - low + 1$$

Then update:

$$new_low = low + \left\lfloor \frac{range \times cum_low(symbol)}{total} \right\rfloor$$

$$new_high = low + \left\lfloor \frac{range \times cum_high(symbol)}{total} \right\rfloor - 1$$

Then set:

$$low = new_low$$

$$high = new_high$$

This selects the part of the current interval belonging to the current symbol.

2.9 Small Worked Example

Suppose the symbols are:

$$A, B, C$$

with counts:

$$A = 5, \quad B = 3, \quad C = 2$$

The total is:

$$10$$

So the ranges are:

$$A \rightarrow [0, 5)$$

$$B \rightarrow [5, 8)$$

$$C \rightarrow [8, 10)$$

For simplicity, suppose the starting interval is:

$$[0, 999]$$

and the message is ABC.

For A:

$$range = 1000$$

$$new_low = 0 + \left\lfloor \frac{1000 \times 0}{10} \right\rfloor = 0$$

$$new_high = 0 + \left\lfloor \frac{1000 \times 5}{10} \right\rfloor - 1 = 499$$

So after A, the interval is:

$$[0, 499]$$

For B:

$$range = 500$$

$$new_low = 0 + \left\lfloor \frac{500 \times 5}{10} \right\rfloor = 250$$

$$new_high = 0 + \left\lfloor \frac{500 \times 8}{10} \right\rfloor - 1 = 399$$

So after B, the interval is:

$$[250, 399]$$

For C:

$$range = 150$$

$$new_low = 250 + \left\lfloor \frac{150 \times 8}{10} \right\rfloor = 370$$

$$new_high = 250 + \left\lfloor \frac{150 \times 10}{10} \right\rfloor - 1 = 399$$

So after C, the interval is:

$$[370, 399]$$

Any number inside this interval represents the message ABC.

2.10 Why Renormalization Is Needed

If the encoder only keeps narrowing the interval, the interval will eventually become too small.

The solution is renormalization.

Renormalization means:

Whenever the current interval tells us a bit that is definitely fixed, write that bit to the payload and then enlarge the remaining interval again.

This is how arithmetic coding can process long files without storing an infinitely precise number.

2.11 The Four Important Boundaries

Use a 32-bit range.

The full interval is:

$$[0, 2^{32} - 1]$$

Define:

$$HALF = 2^{31}$$

$$QUARTER = 2^{30}$$

$$THREE_QUARTER = 3 \times 2^{30}$$

These divide the interval into important parts:

$$[0, QUARTER)$$

$$[QUARTER, HALF)$$

$$[HALF, THREE_QUARTER)$$

$$[THREE_QUARTER, 2^{32})$$

The encoder checks where the current interval lies.

2.12 Renormalization Case 1: Lower Half

If:

$$high < HALF$$

then the whole interval is in the lower half.

That means every number inside the interval starts with binary bit 0.

So the encoder can safely write:

$$0$$

to the payload.

Then the interval is enlarged by shifting it left one bit.

Conceptually, this removes the known leading 0 and continues working with the remaining unknown bits.

2.13 Renormalization Case 2: Upper Half

If:

$$low \geq HALF$$

then the whole interval is in the upper half.

That means every number inside the interval starts with binary bit 1.

So the encoder can safely write:

$$1$$

to the payload.

Before enlarging the interval, subtract *HALF* from both *low* and *high*. This removes the known leading 1.

Then shift the interval left one bit.

Conceptually, this removes the known leading 1 and continues working with the remaining unknown bits.

2.14 Renormalization Case 3: Middle Half

The difficult case is when the interval is not completely in the lower half and not completely in the upper half.

For example, the interval may cross the middle boundary *HALF*.

In that case, the encoder cannot yet decide whether the next output bit should be 0 or 1.

However, if the interval lies inside the middle half:

$$low \geq QUARTER$$

and:

$$high < THREE_QUARTER$$

then the interval is safely away from the extreme lower and upper quarters.

This situation is called the underflow case.

In this case, the encoder does not output a bit immediately.

Instead, it delays the decision.

It increases a counter called:

$$pending_bits$$

by 1.

Then it subtracts *QUARTER* from both *low* and *high*, and enlarges the interval by shifting it left one bit.

The meaning of this is:

We do not yet know the next bit, but we know that when the next definite bit is eventually output, this delayed bit must become the opposite of it.

So if the encoder later outputs a real bit 0, all pending bits become 1.

If the encoder later outputs a real bit 1, all pending bits become 0.

2.15 Outputting a Bit with Pending Bits

Whenever the encoder can finally output a definite bit *b*, it must also output all pending bits.

The rule is:

write *b*

then repeat *pending_bits* times:

write $1 - b$

Then reset:

$$pending_bits = 0$$

For example, if:

$$pending_bits = 3$$

and the encoder finally outputs:

0

then it must actually write:

0 1 1 1

If:

$pending_bits = 3$

and the encoder finally outputs:

1

then it must actually write:

1 0 0 0

2.16 Finishing the Range Encoder

After encoding all input bytes and then EOF, the encoder still has a valid final interval.

It must write enough final bits to make sure the decoder lands inside that final interval.

For this project, after encoding EOF:

1. Increase $pending_bits$ by 1.
2. If $low < QUARTER$, output final bit 0 and then the pending opposite bits.
3. Otherwise, output final bit 1 and then the pending opposite bits.
4. Pad the final byte with zeros if the number of payload bits is not a multiple of 8.

The decoder will stop when it decodes EOF, so the zero padding at the end does not become part of the decompressed file.

2.17 How Decoding Works

The decoder reads the same frequency table as the encoder.

It reconstructs the same cumulative ranges.

Then it reads the first 32 bits of the payload into a variable called:

$code$

If the payload has fewer than 32 bits, the missing bits are treated as zeros.

At each step, the decoder knows:

$low, \quad high, \quad code$

The decoder computes:

$$range = high - low + 1$$

Then it converts $code$ into a cumulative frequency value:

$$value = \left\lfloor \frac{((code - low + 1) \times total - 1)}{range} \right\rfloor$$

This value satisfies:

$$0 \leq \text{value} < \text{total}$$

The decoder finds the symbol whose cumulative range contains this value:

$$\text{cum_low}(\text{symbol}) \leq \text{value} < \text{cum_high}(\text{symbol})$$

That symbol is the next decoded symbol.

After finding the symbol, the decoder narrows the interval using the same equations as the encoder. Then it performs the same renormalization cases:

- If the interval is in the lower half, shift left and read one new payload bit.
- If the interval is in the upper half, subtract *HALF*, shift left, and read one new payload bit.
- If the interval is in the middle half, subtract *QUARTER*, shift left, and read one new payload bit.

The decoder does not need a pending-bit counter. Pending bits are only an encoder-side mechanism for deciding what bits to write.

3 Part 2: Simplified DEFLATE Compressor

In this part, you will implement a simplified compressor inspired by the main ideas inside DEFLATE. The compression pipeline is:

Input Bytes $\rightarrow$ LZ77 Tokens $\rightarrow$ DEFLATE Symbols + Extra Bits $\rightarrow$ Canonical Huffman Bitstream

This is not a full .gz implementation. You are not required to implement the official GZIP header, CRC32, or official DEFLATE block structure.

The final correctness requirement is:

`decompress(compress(data)) = data`

byte-for-byte.

4 Stage 1: LZ77 Pattern Detection

4.1 Purpose

LZ77 finds repeated byte sequences.

Instead of writing the same bytes again, the compressor writes a reference to bytes that already appeared earlier.

For example:

1 `abcabcabcabc`

The first `abc` must be written normally. After that, the remaining `abcabcabc` can be described as:

Go back 3 bytes and copy 9 bytes.

So the LZ77 token stream is:

1 `Literal(97)`
 2 `Literal(98)`
 3 `Literal(99)`
 4 `Match(length=9, distance=3)`

where 97, 98, and 99 are the byte values of `a`, `b`, and `c`.

4.2 Input and Output

The input to Stage 1 is the original byte array:

$$b_0, b_1, b_2, \dots, b_{n-1}$$

where each byte is between 0 and 255.

The output is a list of LZ77 tokens.

There are only two token types:

```
Literal(byte)
```

and:

```
Match(length, distance)
```

A literal means:

Output this byte directly.

A match means:

Go back `distance` bytes in the already reconstructed output and copy `length` bytes.

4.3 Required Constants

Use the following constants:

WINDOW_SIZE = 32768

MIN_MATCH = 3

MAX_MATCH = 258

Therefore, every match must satisfy:

$$3 \leq \text{length} \leq 258$$

and:

$$1 \leq \text{distance} \leq 32768$$

If the best repeated sequence has length 1 or 2, emit literals instead of a match.

4.4 Meaning of Distance

Suppose the decoder has already reconstructed:

1	abc
---	-----

Now it receives:

```
Match(length=9, distance=3)
```

The decoder goes back 3 bytes from the current output position. That points to the start of `abc`. Then it copies 9 bytes.

The result becomes:

1	abcabcabcabc
---	--------------

4.5 Overlapping Matches

LZ77 allows overlapping matches.

For example:

```
1 aaaaaaaaaa
```

can be represented as:

```
1 Literal(97)
2 Match(length=9, distance=1)
```

The decoder first outputs one **a**. Then the match says: go back 1 byte and copy 9 bytes. As the decoder copies new **a** bytes, those newly copied bytes also become available for the rest of the same match.

So the decompressor must copy matches byte by byte.

4.6 Match Selection Rule

At every input position, your compressor must choose the token using this rule:

1. Search only inside the previous 32768 bytes.
2. Find the longest match starting at the current position.
3. If multiple matches have the same length, choose the smaller distance.
4. If the best match length is at least 3, emit a match.
5. Otherwise, emit a literal.

4.7 Simple Hash-Based Match Search

A naive implementation checks every previous position in the last 32768 bytes. This is easy to understand but can be slow.

To make matching faster, use this simple idea:

A valid match must start with at least 3 equal bytes.

So instead of checking all previous positions, group positions by their next 3 bytes.

For example, in:

```
1 abcxxxabcabc
```

the 3-byte sequence **abc** appears at positions 0, 6, and 9.

So when the current position starts with **abc**, it is useful to check only previous positions that also started with **abc**.

4.8 The Dictionary of 3-Byte Keys

Use a dictionary:

`table`

where:

`table[key]` = list of previous positions where this key appeared

The key is simply the next 3 bytes.

For example, if:

```
1 data[i:i+3] == b"abc"
```

then the key may be:

```
1 (97, 98, 99)
```

or the bytes object:

```
1 b"abc"
```

4.9 How the Search Works

At position i :

1. If fewer than 3 bytes remain, emit literals.
2. Otherwise, compute the key `data[i:i+3]`.
3. Look up previous positions stored in `table[key]`.
4. Check candidates from newest to oldest.
5. Ignore candidates where:
$$i - candidate > 32768$$
6. For each remaining candidate, count how many bytes match, up to 258.
7. Choose the longest match.

To keep the search efficient, check at most:

$$\text{MAX_CANDIDATES} = 64$$

previous positions for each current position.

4.10 Updating the Table

Whenever the compressor passes a position, insert its 3-byte key into the table.

If the compressor emits a literal, insert the current position.

If the compressor emits a match of length L , insert all positions covered by that match:

$$i, i + 1, i + 2, \dots, i + L - 1$$

as long as each position has at least 3 bytes remaining.

This helps future matches.

4.11 Stage 1 Requirements

Your Stage 1 implementation must:

1. Treat the input as bytes.
2. Produce only `Literal(byte)` and `Match(length,distance)` tokens.
3. Use the constants `WINDOW_SIZE = 32768`, `MIN_MATCH = 3`, and `MAX_MATCH = 258`.
4. Never emit a match with length less than 3.
5. Never emit a match with length greater than 258.
6. Never emit a match with distance greater than 32768.
7. Support overlapping matches during decompression.
8. Use a dictionary of 3-byte keys, or an equivalent efficient structure, to avoid checking every previous position.

5 Stage 2: DEFLATE Symbols and Extra Bits

5.1 Purpose

Stage 1 produces LZ77 tokens.

Stage 2 converts these tokens into symbols suitable for Huffman coding.

A literal is already a symbol.

A match must be converted into:

length symbol + length extra bits

and:

distance symbol + distance extra bits

5.2 Input and Output

The input to Stage 2 is the LZ77 token stream.

The output is an event stream.

There are three event types:

`LiteralEvent(symbol)`

`MatchEvent(length_symbol, length_extra, distance_symbol, distance_extra)`

`EndEvent(256)`

Here, `length_extra` and `distance_extra` are bit strings.

For example, the length extra bits may be:

01

and the distance extra bits may be:

1

In implementation, these can be represented as strings such as "01" and "1", or as a pair (*value*, *bit_count*). The concept is the same: each extra field is one bit string.

5.3 Literal/Length Symbols

The literal/length alphabet has 286 symbols:

Symbol Range	Meaning
0–255	Literal byte values
256	End-of-block marker
257–285	Length symbols for matches

A literal byte *b* becomes:

`LiteralEvent(b)`

Examples:

$Literal(65) \rightarrow LiteralEvent(65)$

$$Literal(97) \rightarrow LiteralEvent(97)$$

At the end of the event stream, append:

$$EndEvent(256)$$

This end marker tells the decoder where the compressed payload stops.

5.4 Distance Symbols

The distance alphabet has 30 symbols:

$$0, 1, 2, \dots, 29$$

These represent match distances from 1 to 32768.

5.5 Meaning of Extra Bits

Some length and distance symbols represent one exact value.

For example:

$$9 \rightarrow \text{length symbol } 263$$

No extra bits are needed.

Other symbols represent a range.

For example:

$$19, 20, 21, 22 \rightarrow \text{length symbol } 269$$

So symbol 269 only tells the decoder that the length is in the range 19–22.

The extra bits identify the exact value:

$$00 \rightarrow 19$$

$$01 \rightarrow 20$$

$$10 \rightarrow 21$$

$$11 \rightarrow 22$$

Therefore:

$$20 \rightarrow \text{symbol } 269 + \text{extra bits } 01$$

5.6 Length Table

Use the following table exactly.

Length Range	Length Symbol	Extra Bits
3	257	0
4	258	0
5	259	0
6	260	0
7	261	0
8	262	0

9	263	0
10	264	0
11–12	265	1
13–14	266	1
15–16	267	1
17–18	268	1
19–22	269	2
23–26	270	2
27–30	271	2
31–34	272	2
35–42	273	3
43–50	274	3
51–58	275	3
59–66	276	3
67–82	277	4
83–98	278	4
99–114	279	4
115–130	280	4
131–162	281	5
163–194	282	5
195–226	283	5
227–257	284	5
258	285	0

5.7 Length Encoding Arrays

Use these arrays for implementation:

```

1 length_base = [
2     3, 4, 5, 6, 7, 8, 9, 10,
3     11, 13, 15, 17,
4     19, 23, 27, 31,
5     35, 43, 51, 59,
6     67, 83, 99, 115,
7     131, 163, 195, 227,
8     258
9 ]
10
11 length_extra = [
12     0, 0, 0, 0, 0, 0, 0, 0, 0,
13     1, 1, 1, 1,
14     2, 2, 2, 2,
15     3, 3, 3, 3,
16     4, 4, 4, 4,
17     5, 5, 5, 5,
18     0
19 ]

```

For index i , the length symbol is:

$$257 + i$$

To encode a length, find the row where:

$$base \leq length \leq base + 2^{extra_count} - 1$$

Then compute:

$$extra_value = length - base$$

and write that value using exactly `extra_count` bits.

5.8 Distance Table

Use the following table exactly.

Distance Range	Distance Symbol	Extra Bits
1	0	0
2	1	0
3	2	0
4	3	0
5–6	4	1
7–8	5	1
9–12	6	2
13–16	7	2
17–24	8	3
25–32	9	3
33–48	10	4
49–64	11	4
65–96	12	5
97–128	13	5
129–192	14	6
193–256	15	6
257–384	16	7
385–512	17	7
513–768	18	8
769–1024	19	8
1025–1536	20	9
1537–2048	21	9
2049–3072	22	10
3073–4096	23	10
4097–6144	24	11
6145–8192	25	11
8193–12288	26	12
12289–16384	27	12
16385–24576	28	13
24577–32768	29	13

5.9 Distance Encoding Arrays

```

1 distance_base = [
2     1, 2, 3, 4,
3     5, 7,
4     9, 13,
5     17, 25,
6     33, 49,
7     65, 97,
8     129, 193,
9     257, 385,
10    513, 769,
11    1025, 1537,
12    2049, 3073,

```

```

13     4097, 6145,
14     8193, 12289,
15     16385, 24577
16 ]
17
18 distance_extra = [
19     0, 0, 0, 0,
20     1, 1,
21     2, 2,
22     3, 3,
23     4, 4,
24     5, 5,
25     6, 6,
26     7, 7,
27     8, 8,
28     9, 9,
29     10, 10,
30     11, 11,
31     12, 12,
32     13, 13
33 ]

```

For distances, the distance symbol is the array index.

To encode a distance, find the row where:

$$base \leq distance \leq base + 2^{extra_count} - 1$$

Then compute:

$$extra_value = distance - base$$

and write it using exactly `extra_count` bits.

5.10 Stage 2 Example

Stage 1 output:

```

1 Literal(97)
2 Literal(98)
3 Literal(99)
4 Match(length=20, distance=6)

```

The literals become:

LiteralEvent(97), LiteralEvent(98), LiteralEvent(99)

Length 20 belongs to the length range 19–22:

20 → length symbol 269 + extra bits 01

Distance 6 belongs to the distance range 5–6:

6 → distance symbol 4 + extra bits 1

So the match becomes:

MatchEvent(269, 01, 4, 1)

Finally append:

EndEvent(256)

So the Stage 2 event stream is:

```

1 LiteralEvent(97)
2 LiteralEvent(98)
3 LiteralEvent(99)
4 MatchEvent(269, "01", 4, "1")
5 EndEvent(256)

```

6 Stage 3: Canonical Huffman Coding

6.1 Purpose

Stage 3 compresses the symbols produced by Stage 2.

There are two separate Huffman alphabets:

1. Literal/length symbols: 0–285.
2. Distance symbols: 0–29.

Only symbols are Huffman encoded.

Extra bits are not Huffman encoded.

For a match, the payload order is:

Huffman(length_symbol)

then:

length_extra

then:

Huffman(distance_symbol)

then:

distance_extra

6.2 Frequency Counting

Create two frequency arrays:

lit_freq[286]

dist_freq[30]

For each event:

- `LiteralEvent(s)` increases `lit_freq[s]`.
- `MatchEvent(len_sym, len_extra, dist_sym, dist_extra)` increases `lit_freq[len_sym]` and `dist_freq[dist_sym]`.
- `EndEvent(256)` increases `lit_freq[256]`.

Extra bits are not counted because they are raw bits, not Huffman symbols.

6.3 Building Huffman Lengths

For each alphabet, build a normal Huffman tree from frequencies.

The only value you need from the tree is the code length of each symbol.

If a symbol is unused, its length is 0.

If an alphabet contains exactly one used symbol, assign it length 1.

To keep results deterministic, when two nodes have equal frequency, break ties using the smallest symbol value inside the subtree.

6.4 Canonical Huffman Rule

After computing code lengths, convert them to canonical Huffman codes.

Canonical Huffman coding means:

The actual bit codes are determined only by the code lengths and the symbol values.

The rule is:

1. Count how many symbols have each length.
2. Compute the first code for each length.
3. Assign codes to symbols in increasing symbol order.

Use the following code-generation logic:

```
1 count = [0] * 16
2 for length in lengths:
3     count[length] += 1
4
5 count[0] = 0
6
7 next_code = [0] * 16
8 code = 0
9
10 for bits in range(1, 16):
11     code = (code + count[bits - 1]) << 1
12     next_code[bits] = code
13
14 for symbol in range(len(lengths)):
15     length = lengths[symbol]
16     if length != 0:
17         symbol_code[symbol] = next_code[length]
18         next_code[length] += 1
```

6.5 Canonical Example

Suppose the literal/length code lengths are:

Symbol	Length
97	3
98	3
99	2
256	2
263	2

The symbols are ordered by length, then by value:

99, 256, 263, 97, 98

The canonical codes are:

99 → 00

256 → 01

263 → 10

97 → 110

98 → 111

The decompressor can reconstruct these exact codes using only the code lengths.

7 Stage 4: Custom Header and Payload

7.1 Purpose

The compressed payload uses Huffman codes.

Therefore, the decompressor must know the Huffman code lengths before it can decode the payload. Your file format stores:

1. the literal/length code-length table,
2. the distance code-length table,
3. the Huffman-coded payload.

7.2 Compressed File Layout

The compressed file must contain exactly these fields in this order:

Field	Size	Description
LIT_BW	4 bits	Bit-width used for literal/length code lengths
DIST_BW	4 bits	Bit-width used for distance code lengths
LIT_TABLE	$286 \times \text{LIT_BW}$ bits	Code lengths for symbols 0–285
DIST_TABLE	$30 \times \text{DIST_BW}$ bits	Code lengths for symbols 0–29
PAYLOAD	variable	Huffman-coded data plus raw extra bits

7.3 Computing Table Bit-Widths

Let M be the largest code length in a table.

The bit-width is:

$$BW = \begin{cases} 0, & M = 0 \\ \lfloor \log_2(M) \rfloor + 1, & M > 0 \end{cases}$$

Examples:

Maximum length	Bit-width
0	0
1	1
2 or 3	2
4–7	3
8–15	4

LIT_BW is computed from the maximum literal/length code length.

DIST_BW is computed from the maximum distance code length.

If DIST_BW = 0, the distance table has no bits because no distance symbols are used.

7.4 Bit Order

All bits in this assignment are written most-significant bit first.

For example:

$$6 = 110_2$$

Writing 3 bits gives:

$$1, 1, 0$$

A more important example is:

$$2 = 10_2$$

Writing 4 bits gives:

$$0, 0, 1, 0$$

because the value must be padded to exactly 4 bits:

$$0010_2$$

7.5 Payload Writing Rule

After the header, write the event stream.

For:

$$LiteralEvent(s)$$

write:

$$Huffman(s)$$

For:

$$MatchEvent(len_sym, len_extra, dist_sym, dist_extra)$$

write:

$$Huffman(len_sym)$$

then:

$$len_extra$$

then:

$Huffman(dist_sym)$

then:

$dist_extra$

For:

$EndEvent(256)$

write:

$Huffman(256)$

Finally, pad the last byte with zeros if the number of bits is not a multiple of 8.

8 DEFLATE Decompression

8.1 Header Reading

The decompressor first reads:

LIT_BW

and:

$DIST_BW$

Then it reads 286 literal/length code lengths, each using LIT_BW bits.

Then it reads 30 distance code lengths, each using $DIST_BW$ bits.

From these lengths, it reconstructs the canonical Huffman codes.

8.2 Payload Decoding

The decoder repeatedly reads one literal/length Huffman symbol.

There are three cases:

- If the symbol is 0–255, append it to the output as one byte.
- If the symbol is 256, stop.
- If the symbol is 257–285, decode a match.

To decode a match:

1. Use the length table to get the base length and number of length extra bits.
2. Read the length extra bits and compute the actual length.
3. Decode one distance Huffman symbol.
4. Use the distance table to get the base distance and number of distance extra bits.
5. Read the distance extra bits and compute the actual distance.
6. Perform the LZ77 copy into the output buffer.

9 Complete DEFLATE Worked Example

9.1 Input

```
1  abcabcbcabcb
```

Byte values:

97, 98, 99, 97, 98, 99, 97, 98, 99, 97, 98, 99

9.2 Stage 1 Output

```
1  Literal(97)
2  Literal(98)
3  Literal(99)
4  Match(length=9, distance=3)
```

9.3 Stage 2 Output

The literals become:

LiteralEvent(97), LiteralEvent(98), LiteralEvent(99)

Length 9 becomes:

length_symbol = 263

with no extra bits.

Distance 3 becomes:

distance_symbol = 2

with no extra bits.

Append:

EndEvent(256)

So the event stream is:

```
1  LiteralEvent(97)
2  LiteralEvent(98)
3  LiteralEvent(99)
4  MatchEvent(263, "", 2, "")
5  EndEvent(256)
```

9.4 Stage 3 Frequencies

Literal/length frequencies:

Symbol	Frequency
97	1
98	1
99	1
263	1
256	1

Distance frequencies:

Symbol	Frequency
2	1

One valid deterministic Huffman construction gives:

Symbol	Code Length
97	3
98	3
99	2
256	2
263	2

Distance symbol 2 receives length 1.

9.5 Canonical Codes

Literal/length codes:

99 → 00

256 → 01

263 → 10

97 → 110

98 → 111

Distance code:

2 → 0

9.6 Payload Bits

The payload is:

LiteralEvent(97) → 110

LiteralEvent(98) → 111

LiteralEvent(99) → 00

MatchEvent(263, "", 2, "") → 10 0

EndEvent(256) → 01

Therefore:

payload bits = 1101110010001

The final byte is padded with zeros if needed.

10 Report and Comparison

In addition to the source code, you must submit a short report.

The report should explain what you implemented in both parts.

Your report must include:

1. A short explanation of your range coding implementation.
2. A short explanation of your simplified DEFLATE implementation.
3. The file formats used by both compressors.
4. A table comparing compression results on different files.
5. A short discussion of the results.

10.1 Required Test Files

Test your compressors on at least five different files.

Use files such as:

1. A small text file.
2. A larger text file.
3. A file with many repeated characters.
4. A source code file.
5. A random binary file.

10.2 Comparison Table

Your report must contain a table like this:

File	Original Size	Range Size	DEFLATE Size	Best Method
small.txt				
repeated.txt				
code.py				
random.bin				
large.txt				

You may also include compression ratios:

$$\text{Compression Ratio} = \frac{\text{Compressed Size}}{\text{Original Size}}$$

A smaller ratio means better compression.

10.3 Discussion Questions

Answer these questions in your report:

1. Which method compressed repeated text better?
2. Which method compressed normal English text better?
3. What happened with random data?
4. Why can small files become larger after compression?

5. Why does DEFLATE usually benefit from LZ77 before Huffman coding?
6. Why does range coding need a frequency table?
7. How does the decoder know when to stop in each method?

11 Submission Format

Submit a single `.zip` file containing all source code files needed to run your compressors and decompressors.

The submitted folder must include a runnable file named:

```
1 main.py
```

Your program must support the following terminal arguments.

11.1 Part 1 Commands: Range Coding

To compress a file using range coding:

```
1 python main.py -rc filename
```

This should create the compressed file in the same directory as the input file.

The compressed output file must be named by appending `.rng` to the original filename.

For example, compressing:

```
1 data.txt
```

must produce:

```
1 data.txt.rng
```

To decompress a range-coded file:

```
1 python main.py -rd filename
```

For example:

```
1 python main.py -rd data.txt.rng
```

should produce:

```
1 data.txt
```

11.2 Part 2 Commands: Simplified DEFLATE

To compress a file using the simplified DEFLATE compressor:

```
1 python main.py -c filename
```

This should create the compressed file in the same directory as the input file.

The compressed output file must be named by appending `.sdf1` to the original filename.

For example, compressing:

```
1 data.txt
```

must produce:

```
1 data.txt.sdf1
```

To decompress a simplified DEFLATE file:

```
1 python main.py -d filename
```

For example:

```
1 python main.py -d data.txt.sdf1
```

should produce:

```
1 data.txt
```

11.3 Required Submitted Files

Your submitted folder must include:

1. Your source code.
2. The runnable `main.py` file.
3. Any helper files needed to run the project.
4. A report file named `report.pdf`